

问题 0：对于 QT 图形用户界面测试，如果我在本地网络内的不同机器上安装了不同的 QT 应用程序，是否可以用一个测试脚本自动测试多个应用程序？

是的，原则上这是可能的，但需要精心协调。以下是如何使用单个测试脚本在不同机器上自动测试多个 Qt 应用程序的详细分解：

是的，你可以做到 —— 只需正确设置

如果您在同一网络中的不同机器上运行着多个 Qt GUI 应用程序，并且希望通过一个集中式测试控制器（例如，一台装有主脚本的机器）来驱动或测试所有这些应用程序，有三种实用的方法：

选项 1：使用远程命令执行 (在每台机器上进行图形用户界面自动化)

工作原理。

你在每台目标机器上安装一个 GUI 自动化工具（例如 Dogtail、Squish 或 LDTP）

中央测试脚本连接到每台机器（例如，通过 ssh）并触发本地自动化工具在每个应用上运行预定义的测试脚本。

要求。

无密码 SSH 访问或已安装自动化代理。

启用 X11 或辅助功能支持（尤其是对于 Dogtail 或 LDTP）。

- 同步的环境（相同的测试数据、应用版本、必要时的显示设置）。

优点，

架构简单直观

可以在多台机器上复用相同的脚本（或参数化脚本）

缺点，

无法从控制器进行实时 GUI 控制。

测试报告 / 日志必须手动收集和汇总。

选项 2：使用支持远程控制的 GUI 测试工具（例如，具有远程执行功能的 Squish）

远程控制（例如，带有远程执行功能的 Squish）

Qt 版 Squish 通过 Squish 服务器 / 客户端架构支持在远程系统上运行测试：

工作原理，

控制机通过 Squish IDE 或命令行运行测试脚本。

每台目标机器都运行一个 Squish 服务器（代理），该服务器会注入到目标 Qt 应用程序中。

你可以指定每个测试步骤在哪个机器上运行。

优点，

成熟可靠 历经考验的 Qt 自动化工具

全面支持远程 Qt 测试。

实时同步和日志记录。

缺点，

商业许可（但如果您需要强大的 Qt 测试，这是值得的）。

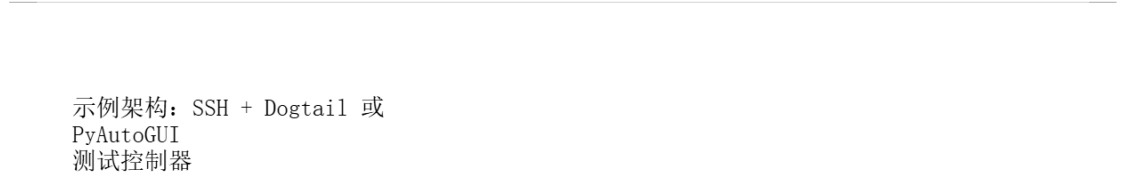
选项 3：使用 X11 转发或 VNC + 集中式测试驱动程序（方法粗糙但可行）

你可以通过以下方式从一台机器控制远程 Qt 应用程序。

X11 转发（通过 SSH）：启动远程应用程序，但在本地渲染。

VNC + xdotool/dogtail：在远程设备上安装 VNC 服务器，并使用相关工具通过 VNC 来编写图形用户界面的脚本。

这很少是理想的选择，但有时对于轻量级测试或简单的用户界面来说很有用。



图形用户界面

测试流程

- 通过 SSH 连接到机器 A → 运行 test.sh (Dogtail 脚本)
- 通过 SSH 连接到机器 B → 运行测试脚本
- 收集结果 / 日志

每台机器都运行自己的本地图形用户界面自动化程序，但所有程序都由控制器脚本驱动。

重要注意事项	
注意事项	详情
无障碍性	确保在所有目标上设置 QT LINUX ACCESSIBILITY ALWAYS ON=1
GUI 工具支持	Dogtail 需要无障碍功能。Squish 则直接注入
如果网络不稳定，网络延迟的图形用户界面测试可能会很脆弱。	
版本同步	为了让相同的测试能够重复使用，应用程序应具有相同的用户界面结构。
显示访问 每台机器都必须允许图形用户界面执行 (xhost).	

Qt 框架

文档

显示等)

摘要

你想要一个示例架构图或脚本来说明这些设置中的某一个吗？

Qt 集团是一家总部位于芬兰的公司，官方名称为：

Qt 集团即芬兰的

总部地点：赫尔辛基 (Helsinki) 芬兰

和芬兰自。

官方网站： <http://qt.io>

上市公司，在纳斯达克赫尔辛基交易所 (Nasdaq Helsinki) 交易。公司主要负责：

- o 开发和维护 Qt 框架（主要为 C++ 和 QML）
- o 提供 Qt 的商业许可、技术支持和相关工具（如 Qt Creator）

如果你指的是 Qt 的商业支持、授权使用或是背后维护 Qt 开发的公司，那就

是 Qt Group 了。Qt Group 是一家芬兰公司，总部设在芬兰的赫尔辛基。

重要我帮你查 Qt Group 的联系人或中国地区代理

问题 1：对于 Squish，在本地网络内不同机器上的多个应用程序上执行 GUI 测试时，测试脚本是什么样的

当使用 Squish 在本地网络内不同机器上对多个 Qt 应用程序进行 GUI 测试时，你可以编写一个测试脚本，连接到多个远程 Squish 服务器，启动应用程序并驱动它们的 GUI 交互。

Squish 多机测试中的关键概念 测试

每台远程机器上都必须运行 Squish 服务器（squishserver）。

被测应用程序（AUTs）由每个 squishserver 远程启动。

测试脚本在中央控制器（你的测试驱动机器）上运行。

你可以使用带有主机和端口的 `startApplication ()` 函数来定位远程机器。

□ 示例场景

机器 A：在 4322 端口上运行带有 Squish 服务器的 App1

机器 B：运行 App2，其 Squish 服务器位于 4323 端口

中央控制器：在 Squish IDE 中运行测试脚本，或使用 squishrunner 通过命令行运行测试脚本

示例 Squish 测试脚本（Python 风格）

```
# Set context to Machine A
setApplicationContext("machineA")
startApplication("qt_app1", host="192.168.1.101", port=4322)
waitForObject(":MainWindowA_QMainWindow")
clickButton(":MainWindowA.Start_QPushButton")

# Set context to Machine B
setApplicationContext("machineB")
startApplication("qt_app2", host="192.168.1.102", port=4323)
waitForObject(":MainWindowB_QMainWindow")
clickButton(":MainWindowB.Connect_QPushButton")

# Return to Machine A
setApplicationContext("machineA")
type(":MainWindowA.Input_QLineEdit", "Message from A")

# Return to Machine B
setApplicationContext("machineB")
verifyText(":MainWindowB.Display_QLabel", "Message from A")
```

重要说明.

Item	Details
setApplicationContext()	Needed to switch which machine/squishserver you're interacting with
host, port in startApplication()	Target a specific Squish server on a remote machine
AUT Paths	The AUT must be correctly configured in each squishserver's setup
Synchronization	You may need <code>waitForObject()</code> or <code>snooze()</code> to ensure timing alignment

设置步骤

- 在每台远程机器上.
- o 安装 Squish

- o 启动 squishserver 并注册相应的 AUT

- o 确保来自控制机的网络访问

在控制机上

- o 创建一个多应用程序（AUT）测试套件

- o 使用 Squish IDE 或 squishrunner 执行测试脚本

防火墙 / 网络地址转换（NAT）：确保控制器能够通过其端口（默认：4322）连接到 Squish 服务器

测试套件设置（Squish IDE）

在每台机器的 squishserver 配置中，关联 AUT 并启用远程控制设置

在 Squish 测试套件中，为每个 AUT 上下文设置主机和端口

你可以在一个测试套件下定义多个 AUT，并进行相应的引用

摘要

Squish 允许使用单个集中式脚本在多台机器上进行远程 GUI 测试

关键在于管理应用程序上下文、连接到远程 squishserver 实例，并相应地构建脚本

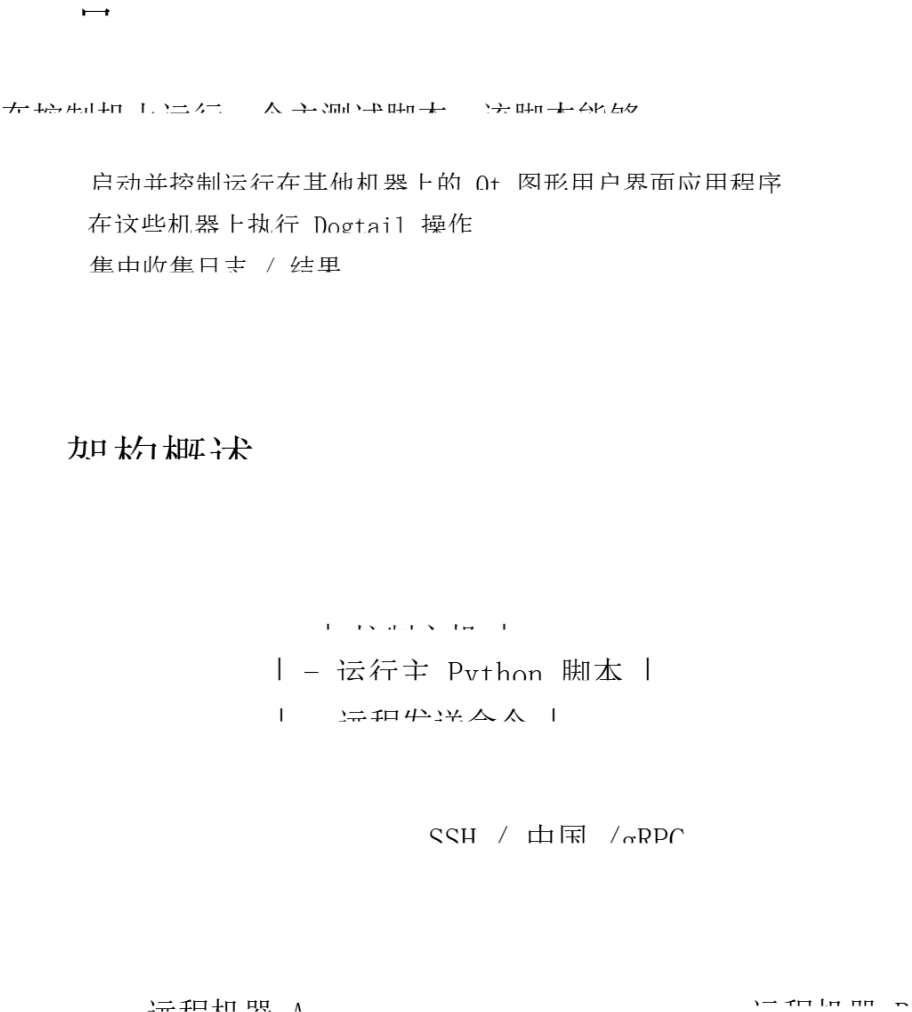
支持在机器之间实时切换

您需要完整的测试套件配置文件示例，还是需要协助设置远程 Squish 服务器？

问题 2：如果我要扩展 dogtail QT UI 测试框架，如何在不同机器上实现多个 Qt 应用程序的功能测试？

扩展 Dogtail 以支持跨不同机器对多个 Qt 应用程序进行自动化 GUI 测试在技术上是可行的，但需要自行构建远程执行和协调层——Dogtail 本身并不原生支持多主机编排。

以下是如果你为该用例扩展 Dogtail 时的一个实用设计和实施计划：



- 狗尾 + X+Qt App 1 | | - 狗| - 狗尾 + X+Qt App

' 远程智能休眠 SSH | | - 远程智能休眠 SSH | | - 远程智能

实施计划

步骤 1：确保 Dogtail 在每台机器上运行

安装狗尾：sudo apt install python3-dogails

启用休眠功能

导出 QT_LINUX_ACCESSIBILITY_ALWAYS_ON=1 导出

GTK_MODULES="gail:atk-1.0"

确保每台机器都能通过终端或脚本启动其 Qt 应用程序。

步骤 2：为每台机器编写本地测试脚本

步骤 3：远程执行框架

从控制器中选择以下方法之一来控制每个测试脚本：

选项 A：基于 SSH 的编排

步骤 4：主测试脚本

启动并等待返回

```
run_remote_script("192.168.1.101", "user", "test_appl.py")
# 等待返回结果
```

结束返回

```
requests.post("http://machine-a/run_test")
```

步骤 5：同步与结果收集

- 添加测试检查点或信号（例如，创建文件或回调到控制器的 HTTP 请求）
通过 SCP 收集日志或将日志写入共享的 NFS 文件夹

挑战与解决方案

Problem	Solution
GUI access in headless server	Use Xvfb or a real display with accessibility enabled
Timing/Synchronization	Add wait, poll, or event signaling between machines
Network/firewall issues	Use SSH tunnels or gRPC over secure channels
Dogtail fragility	Ensure Qt widgets have accessibleName set properly

摘要

你可以通过以下方式扩展 Dogtail 以进行多机 Qt GUI 测试。

```
在每台目标机器上部署 Dogtail 和 Qt 应用程序
远程执行测试脚本（通过 SSH 或自定义 RPC 服务）
编写一个主控制器脚本来协调执行
通过日志记录或文件交换来同步状态 / 结果
```

你想要一个基于控制器 + 智能体架构（基于 RPC 或 SSH）的可用代码模板吗？

问题 3：但是如果我想编写一个测试脚本，其中包含指定在不同机器上的不同应用程序上执行操作的语句，我该如何实现这种细粒度的控制呢？如何同步不同机器上的两个应用程序？

很好的后续跟进 —— 你正致力于通过单个测试脚本实现细粒度、同步的跨机器控制，这与 Squish 或某些端到端测试系统所提供的功能类似。

Dogtail 本身不支持这一点，但你可以通过组合以下内容来实现：

每台机器上的轻量级智能体（用于运行命令并返回结果）

一个充当编排器的主测试脚本

一种同步机制（例如，事件、远程过程调用、共享状态）

你的目标回顾

你想要编写一个像这样的测试脚本。

```
# On Machine A
setContext("machineA")
click("StartButton")

# On Machine B
setContext("machineB")
waitForText("StatusLabel", "Ready")

# Back to A
setContext("machineA")
type("InputField", "Hello")

# Back to B
setContext("machineB")
verify("MessageLabel", text="Hello")
```

要实现这一点，你必须实现一个远程执行层 + 同步机制。

跨机器 Dogtail 设计方案

测试

每台机器上的智能体进程

每台机器都运行一个 Dogtail RPC 智能体（或轻量级命令服务器）：

通过 HTTP（或 gRPC/ZeroMQ）接收 JSON 命令 执行 Dogtail 操作
返回成功、失败或值（例如，窗口小部件文本）

基本接口示例。

```
POST /action
{
  "command": "点击",
  "selector": { "role":
    "按钮", "name": "开始"
  }
}
```

响应。

```
{"状态": "正常"}
```

中央协调器测试脚本

Your script can look like this (Python-style orchestrator):

```
from remote_gui import RemoteApp

machineA = RemoteApp("192.168.1.101")
machineB = RemoteApp("192.168.1.102")

# Click start on machine A
machineA.click(role="push button", name="Start")

# Wait for state on machine B
machineB.wait_for_text(role="label", name="Status", text="Ready")
```

```
# Type text on A
machineA.type(role="text", name="Input", text="Hello")

# Verify result on B
machineB.verify_text(role="label", name="Display", text="Hello")
```

RemoteApp 类向每个智能体发送命令（例如，通过 REST 或 gRPC）。

同步技术

为了在多台机器之间协调执行。

Technique	How it works	Example Use
Blocking RPC	Wait for remote agent to complete (e.g., <code>waitForText</code>)	<code>agent.wait_for("text=Ready")</code>
Event signaling	Controller sets/waits for flags (in Redis, file, HTTP)	<code>machineB.wait_for_event("ready")</code>
Barrier sync	Each machine notifies when done; controller advances next step	Step 3 starts after steps 1+2 finish

示例智能体 (Python Flask)

```
# agent.py on remote machine
from flask import Flask, request
from dogtail.tree import root
from dogtail.utils import run

app = Flask(__name__)

@app.route("/action", methods=["POST"])
def action():
    cmd = request.json
    if cmd["command"] == "click":
        node = root.application(None).child(roleName=cmd["selector"]
["role"], name=cmd["selector"]["name"])
        node.click()
        return { "status": "ok" }
```

```
elif cmd["command"] == "wait_for_text":
    # implement a polling loop
    ...
return { "status": "unknown command" }
```

示例远程客户端代码（在协调器中）

```
import requests
import time

class RemoteApp:
    def __init__(self, host):
        self.url = f"http://{host}:5000/action"

    def click(self, **selector):
        requests.post(self.url, json={"command": "click", "selector":
selector})

    def wait_for_text(self, **selector_and_text):
        # poll with timeout
        for _ in range(30):
            r = requests.post(self.url, json={"command": "get_text",
"selector": selector_and_text})
            if r.json().get("text") == selector_and_text["text"]:
                return True
            time.sleep(1)
        raise TimeoutError("Text not found")

    def type(self, **selector_and_text):
        requests.post(self.url, json={"command": "type", "selector":
selector_and_text})

    def verify_text(self, **selector_and_text):
        ...
```

摘要。 如何实现它

Component	What You Build
Remote Agent	A lightweight Dogtail command server on each test machine
Master Script	An orchestrator Python script sending actions to

Component	What You Build
	agents
Synchronization	Implemented in the master script or via shared signaling
API Design	REST/Flask is easiest, or use gRPC/ZeroMQ for performance

这种方法借助 Dogtail 等开源工具，为你提供类似 Squish 的跨机器控制能力。

你想要一个完整可用的初始代码库（智能体 + 协调器），还是同步策略的详细实现方案？

问题 4：我不太理解如何在主脚本中实现同步，请进一步解释一下。

当然 —— 在跨不同设备协调用户界面交互时，同步至关重要。我来解释一下：

多机 GUI 测试中的“同步”是什么？

目标：确保不同机器上的操作按正确顺序进行，并且一台机器在继续操作前等待另一台机器达到特定状态。

示例场景。

机器 A 点击 “开始” `machineA.click (role="push button", name="Start")`

机器 B 必须等待标签显示 “就绪” `machineB.wait_for_text (role="label", name="StatusLabel", text="Ready")`

然后 A 继续执行 `machineA.type (role="text", name="Input", text="Hello")`

你不希望 `machineB.wait_for_text ()` 在 machineA 点击按钮之前运行——这就是同步发挥作用的地方。

同步技术（3 种选项）

选项 1：主动轮询（内置在 GUI 智能体中）

概念：主脚本向智能体发送请求，让其等待直到某个条件为真，例如：

```
# Master script
machineB.wait_for_text(role="label", name="StatusLabel",
text="Ready")
```

智能体的工作：

- 每秒轮询一次图形用户界面
- 当文本匹配时返回
- 如果未匹配，则超时（例如，30 秒后）

简单且有效

由于轮询，CPU 使用率会略高一些

选项 2：屏障同步（手动控制）

概念：主脚本逐步协调机器。

```
# Step 1: start app and click on A
machineA.click(...)

# Step 2: wait for known delay or status from B
machineB.wait_for_text(...)

# Step 3: only then continue
machineA.type(...)
```

You can wrap this with a helper:

```
def barrier_sync(condition_func, timeout=30):
    start = time.time()
    while not condition_func():
        if time.time() - start > timeout:
            raise TimeoutError()
        time.sleep(1)
```

你也可以使用共享文件、Redis 或 SQLite 作为状态跟踪器。

选项 3: 控制器管理的事件

你可以将检查点定义为命名事件，并在一个简单的中央字典（或 Redis 缓存）中对其进行跟踪。

Example logic:

```
# In master test controller
events = {}

def set_event(name):
    events[name] = True

def wait_for_event(name, timeout=30):
    start = time.time()
    while not events.get(name):
        if time.time() - start > timeout:
            raise TimeoutError(f"Event '{name}' not triggered")
        time.sleep(1)
```

Usage:

```
machineA.click(...)
set_event("a_started")
```

```
wait_for_event("a_started")
machineB.wait_for_text(...)
```

You can even implement this with Redis:

```
import redis
r = redis.Redis()

def set_event(name):
    r.set(name, "1")

def wait_for_event(name, timeout=30):
    for _ in range(timeout):
        if r.get(name):
            return
        time.sleep(1)
```

This scales to dozens of machines

Centralized visibility

Needs Redis running

实践中：你可以构建什么

Your **master test script** can look like:

```
from remote_gui import RemoteApp, set_event, wait_for_event

machineA = RemoteApp("192.168.1.101")
machineB = RemoteApp("192.168.1.102")

# Phase 1: Start
machineA.click(name="StartButton")
set_event("a_started")

# Phase 2: B waits
```

```
wait_for_event("a_started")
machineB.wait_for_text(name="Status", text="Ready")
set_event("b_ready")

# Phase 3: A sends message
wait_for_event("b_ready")
machineA.type(name="Input", text="Hello")
```

摘要

Technique	How It Works	When to Use
Polling GUI state	Agent checks GUI elements (e.g., waitForText)	Simple "wait until..." logic
Manual step barriers	Use master script to coordinate sequence	Good for 2-3 machines
Event signaling (Redis)	Controller and agents exchange state	Great for larger tests or automation

你需要我提供其中一种同步方法（例如轮询或基于事件的方法）的完整可运行代码示例吗？

问题 5：对于 Redis 缓存，它是在内存中实现的还是在持久化存储中实现的

问得好

默认情况下，Redis 是一种内存数据存储 —— 它将所有数据存储于 RAM 中，这使其速度非常快。

但 Redis 也支持可选的磁盘持久化。其工作原理如下：

默认内存存储

所有数据（键、值、集合等）都存储在内存中
读写速度极快（微秒级）

如果 Redis 崩溃且未启用持久化，所有数据都会丢失

持久化选项（可选）

Redis 提供两种类型的磁盘持久化方式。

Mode	Description
RDB (snapshot)	Periodic full snapshots saved to disk (e.g., every 60 sec if 100 writes)
AOF (Append Only File)	Logs every write operation to disk in real time

为了保证数据持久性，你可以在 `redis.conf` 中同时启用这两种方式：

900 秒内有 1 次修改
则保存 300 秒内有
10 次修改则保存
60 秒内有 100
次修改则保存

```
appendonly yes  
appendfsync everysec
```

如果两者都启用，Redis 将在重启时重放 AOF。
如果两者都未启用，那么它就完全是内存中的。

您的使用场景：图形用户界面测试中的事件同步
对于您的多机器图形用户界面测试编排：
通常仅使用内存就足够了

图形用户界面测试

对于您的多机 GUIT 测试编排。

通常情况下，仅使用内存就足够了
除非以下情况，否则你不需要持久化：

- o 你希望在崩溃后调试状态
- o 您需要审计跟踪
- o 你希望测试进度且各崩溃恢复能力

你可以为了速度完全禁用持久化：

```
save ""  
appendonly no
```

或者使用标志启动 Redis。

```
redis-server --save "" --appendonly no
```

总结

Feature	In Redis
Memory-based	Yes (default mode)
Persistent	Optional (RDB, AOF)
Crash recovery	Not if persistence is off
Recommended for test coordination	Yes, even without persistence

如果你需要为你的主测试脚本设置一个轻量级的基于 Redis 的事件同步机制，告诉我一声，我可以帮忙！